

Experiment No-3

Student Name: HARSH

Branch: MCA

Semester: 3rd

Subject Name: Machine Learning Lab

UID: 24MCA20045

Section/Group: 24MCA-1(B)

Date of Performance: 16/09/2025

Subject Code: 24CAP-702

Aim: Use MongoDB for performing CRUD operations on Faculty Management System.

Task to be done:

The objective of using Mongoose to connect to a MongoDB database and perform CRUD operations is to:

1. Establish a connection between a Node.js application and a MongoDB database.
2. Provide a structured and schema-based way to define data models.
3. Perform CRUD operations (Create, Read, Update, Delete) on MongoDB data easily and efficiently.
4. Simplify database interactions by using high-level JavaScript methods instead of raw MongoDB queries.
5. Ensure data validation and consistency using Mongoose's built-in schema validation.

Flowchart:

Step 1: Install and Import Mongoose

Step 2: Connect to MongoDB

Step 3: Define a Schema

Step 4: Create a Model

Step 5: Perform CRUD Operations

A. Create (Insert Data)

B. Read (Fetch Data)

C. Update (Modify Existing Data)

D. Delete (Remove Data)

Step 6: Handle Errors and Close Connection

Code:

models/personal.js:

```
const mongoose = require('mongoose');
```

```
const personalSchema = new mongoose.Schema({
```

```
  employeeId: {  
    type: String,  
    required: true,  
    unique: true
```

```
},
firstName: {
  type: String,
  required: true
},
lastName: {
  type: String,
  required: true
},
email: {
  type: String,
  required: true,
  unique: true
},
phone: String
});
module.exports = mongoose.model('PersonalDetail', personalSchema);
```

models/professional.js:

```
const mongoose = require('mongoose');

const professionalSchema = new mongoose.Schema({
  personalId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: 'PersonalDetail'
  },
  employeeId: {
    type: String,
    required: true,
    unique: true
  },
  department: String,
  jobTitle: String,
  dateOfJoining: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('ProfessionalDetail', professionalSchema);
```

app.js:

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
```

```
// Routes
const personalRoutes = require('./routes/personalRoutes');
const professionalRoutes = require('./routes/professionalRoutes');

const app = express();
const PORT = 5000;

// Middleware
app.use(bodyParser.json());

// MongoDB connection
mongoose.connect('mongodb://localhost:27017/employee_linked_db', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

// Routes
app.use('/personal', personalRoutes);
app.use('/professional', professionalRoutes);

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Routes/personalRoutes.js

```
const express = require('express');
const router = express.Router();
const Personal = require('../models/personal');
const Professional = require('../models/professional');

// CREATE Personal Record
router.post('/', async (req, res) => {
  try {
    const newPersonal = new Personal(req.body);
    const savedPersonal = await newPersonal.save();
    res.status(201).json(savedPersonal);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// READ Personal Record by employeeId
```

```
router.get('/:employeeId', async (req, res) => {
  try {
    const personal = await Personal.findOne({ employeeId: req.params.employeeId });
    if (!personal) return res.status(404).json({ message: "Not found" });
    res.json(personal);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

// HEAD - Check if record exists

```
router.head('/:employeeId', async (req, res) => {
  try {
    const personal = await Personal.findOne({ employeeId: req.params.employeeId });
    if (!personal) return res.status(404).end();
    res.status(200).end();
  } catch (err) {
    res.status(500).end();
  }
});
```

// UPDATE (Full) Personal Record by employeeId

```
router.put('/:employeeId', async (req, res) => {
  try {
    const updated = await Personal.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      req.body,
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

// PATCH (Partial Update)

```
router.patch('/:employeeId', async (req, res) => {
  try {
    const updated = await Personal.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      { $set: req.body },
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
```

```
res.status(400).json({ message: err.message });
    }
});

// DELETE Personal Record (and linked professional)
router.delete('/:employeeId', async (req, res) => {
  try {
    const personalDoc = await Personal.findOneAndDelete({ employeeId: req.params.employeeId });
    if (!personalDoc) return res.status(404).json({ message: "Not found" });

    await Professional.deleteOne({ personalId: personalDoc._id });
    res.json({ message: "Personal and linked professional records deleted" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// OPTIONS
router.options('/', (req, res) => {
  res.set('Allow', 'GET,HEAD,POST,PUT,PATCH,DELETE,OPTIONS');
  res.sendStatus(200);
});

module.exports = router;
```

Routes/professionalRoutes.js

```
const express = require('express');
const router = express.Router();
const Personal = require('../models/personal');
const Professional = require('../models/professional');

// CREATE Professional Record
router.post('/', async (req, res) => {
  try {
    const personalDoc = await Personal.findOne({ employeeId: req.body.employeeId });
    if (!personalDoc) return res.status(404).json({ message: "Personal record not found. Create it first." });

    const newProfessional = new Professional({
      ...req.body,
      personalId: personalDoc._id
    });
    const savedProfessional = await newProfessional.save();
    res.status(201).json(savedProfessional);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

```
// READ Professional Record (with personal details) by employeeId
router.get('/:employeeId', async (req, res) => {
  try {
    const professional = await Professional.findOne({ employeeId: req.params.employeeId
  }).populate('personalId');
    if (!professional) return res.status(404).json({ message: "Not found" });
    res.json(professional);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// HEAD - Check if record exists
router.head('/:employeeId', async (req, res) => {
  try {
    const professional = await Professional.findOne({ employeeId: req.params.employeeId });
    if (!professional) return res.status(404).end();
    res.status(200).end();
  } catch (err) {
    res.status(500).end();
  }
});

// UPDATE (Full) Professional Record
router.put('/:employeeId', async (req, res) => {
  try {
    const updated = await Professional.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      req.body,
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// PATCH (Partial Update)
router.patch('/:employeeId', async (req, res) => {
  try {
    const updated = await Professional.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      { $set: req.body },
      { new: true }
    );
  }
});
```

```

    });
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// DELETE Professional Record
router.delete('/:employeeId', async (req, res) => {
  try {
    const deleted = await Professional.findOneAndDelete({ employeeId: req.params.employeeId });
    if (!deleted) return res.status(404).json({ message: "Not found" });
    res.json({ message: "Professional record deleted" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

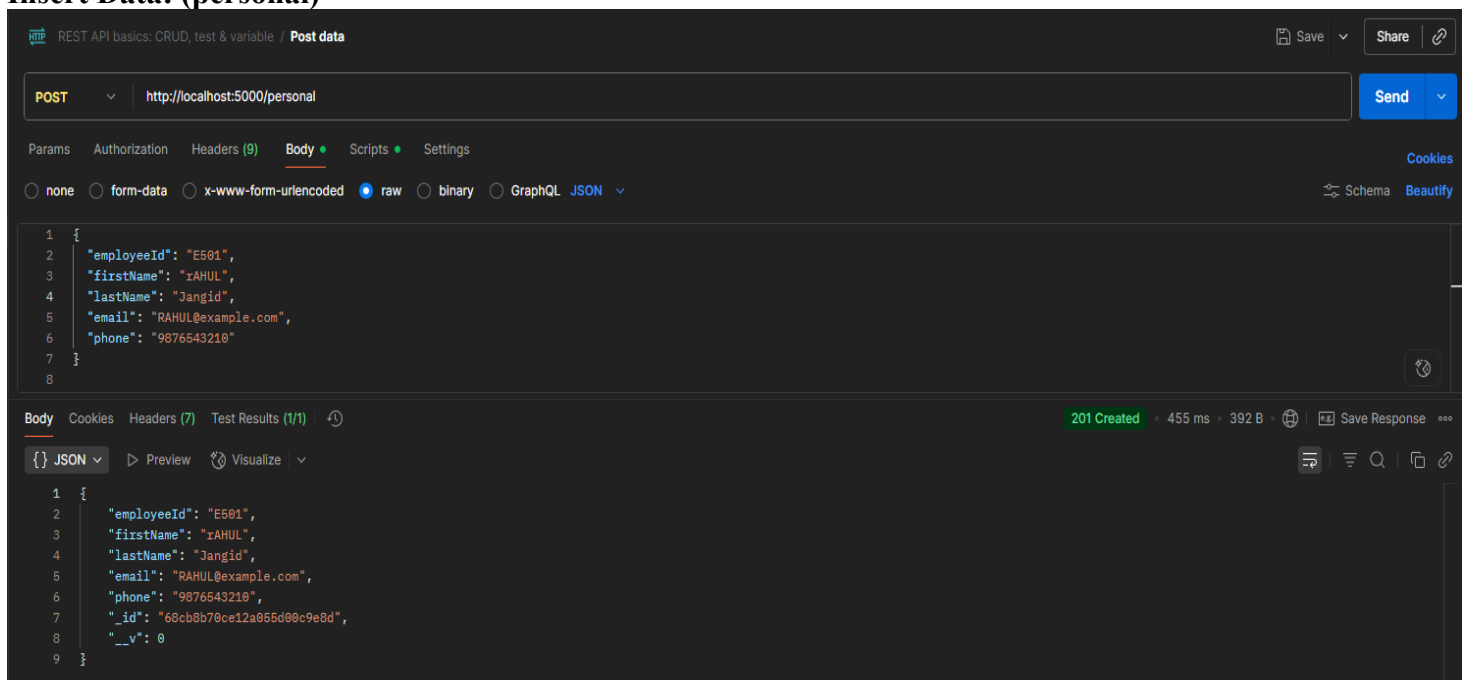
// OPTIONS
router.options('/', (req, res) => {
  res.set('Allow', 'GET,HEAD,POST,PUT,PATCH,DELETE,OPTIONS');
  res.sendStatus(200);
});

module.exports = router;

```

Output:

Insert Data: (personal)



REST API basics: CRUD, test & variable / Post data

POST http://localhost:5000/personal

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "employeeId": "E581",
3   "firstName": "RAHUL",
4   "lastName": "Jangid",
5   "email": "RAHUL@example.com",
6   "phone": "9876543218"
7 }
8

```

Body Cookies Headers (7) Test Results (1/1)

201 Created • 455 ms • 392 B • Save Response

JSON Preview Visualize

```

1 {
2   "employeeId": "E581",
3   "firstName": "RAHUL",
4   "lastName": "Jangid",
5   "email": "RAHUL@example.com",
6   "phone": "9876543218",
7   "_id": "68cbb780ce12a065d08c9e8d",
8   "__v": 0
9 }

```



(professional)

REST API basics: CRUD, test & variable / Post data

POST http://localhost:5000/professional

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "employeeId": "E501",
3   "department": "frontend",
4   "jobTitle": "Software Engineer"
5 }
6
```

Body Cookies Headers (7) Test Results (1/1) 201 Created 65 ms 441 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "personalId": "68cb8b70ce12a055d00c9e8d",
3   "employeeId": "E501",
4   "department": "frontend",
5   "jobTitle": "Software Engineer",
6   "_id": "68cb8be7ce12a055d00c9e91",
7   "dateOfJoining": "2025-09-18T04:34:47.011Z",
8   "__v": 0
9 }
```

Fetching Data(personal):

REST API basics: CRUD, test & variable / Get data

GET http://localhost:5000/personal/E501

Params Authorization Headers (7) Body Scripts Settings

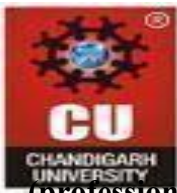
none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 Ctrl+Alt+P for Postbot

Body Cookies Headers (7) Test Results

{ } JSON Preview Visualize

```
1 {
2   "_id": "68cb8b70ce12a055d00c9e8d",
3   "employeeId": "E501",
4   "firstName": "RAHUL",
5   "lastName": "Jangid",
6   "email": "RAHUL@example.com",
7   "phone": "9876543210",
8   "__v": 0
9 }
```

(professional):

REST API basics: CRUD, test & variable / **Get data**

GET

Params Authorization Headers (7) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

1 Ctrl+Alt+P for Postbot

Body Cookies Headers (7) Test Results

{ } **JSON** Preview Visualize

```
1 {
2   "_id": "68cb8be7ce12a055d00c9e91",
3   "personalId": {
4     "_id": "68cb8b70ce12a055d00c9e8d",
5     "employeeId": "E501",
6     "firstName": "RAHUL",
7     "lastName": "Jangid",
8     "email": "RAHUL@example.com",
9     "phone": "9876543210",
10    "__v": 0
11  },
12  "employeeId": "E501",
13  "department": "frontend",
14  "jobTitle": "Software Engineer",
15  "dateOfJoining": "2025-09-18T04:34:47.011Z",
16  "__v": 0
17 }
```

Modifying Data by Id with PUT Request:

(personal)

REST API basics: CRUD, test & variable / **Post data**

PUT

Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "firstName": "Harsh",
3   "lastName": "Jangid",
4   "email": "Harsh@example.com",
5   "phone": "9050432145"
6 }
```

Body Cookies Headers (7) Test Results (1/1)

{ } **JSON** Preview Visualize

```
1 {
2   "_id": "68cb8b70ce12a055d00c9e8d",
3   "employeeId": "E501",
4   "firstName": "Harsh",
5   "lastName": "Jangid",
6   "email": "Harsh@example.com",
7   "phone": "9050432145",
8   "__v": 0
9 }
```

(professional):

REST API basics: CRUD, test & variable / **Post data**

PUT ▼ http://localhost:5000/professional/E501

Params Authorization Headers (9) **Body** • Scripts • Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "department": "backend",
3   "jobTitle": "java developer"
4 }
5
```

Body Cookies Headers (7) Test Results (1/1) ↻

{} **JSON** ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "_id": "68cb8be7ce12a055d00c9e91",
3   "personalId": "68cb8b70ce12a055d00c9e8d",
4   "employeeId": "E501",
5   "department": "backend",
6   "jobTitle": "java developer",
7   "dateOfJoining": "2025-09-18T04:34:47.011Z",
8   "__v": 0
9 }
```

Modifying Data by Id with PATCH Request:

REST API basics: CRUD, test & variable / **Post data**

PATCH ▼ http://localhost:5000/personal/E501

Params Authorization Headers (9) **Body** • Scripts • Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "firstName": "Rahul"
3 }
```

Body Cookies Headers (7) Test Results (1/1) ↻

{} **JSON** ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "_id": "68cb8b70ce12a055d00c9e8d",
3   "employeeId": "E501",
4   "firstName": "Rahul",
5   "lastName": "Jangid",
6   "email": "Harsh@example.com",
7   "phone": "9050432145",
8   "__v": 0
9 }
```



Professional:

REST API basics: CRUD, test & variable / **Post data**

PATCH `http://localhost:5000/professional/E501`

Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "department": "HR"
3 }
4
```

Body Cookies Headers (7) Test Results (1/1)

{} **JSON** Preview Visualize

```
1 {
2   "_id": "68cb8be7ce12a055d00c9e91",
3   "personalId": "68cb8b70ce12a055d00c9e8d",
4   "employeeId": "E501",
5   "department": "HR",
6   "jobTitle": "java developer",
7   "dateOfJoining": "2025-09-18T04:34:47.011Z",
8   "__v": 0
9 }
```

HEAD req:

REST API basics: CRUD, test & variable / **Post data** Save Share

HEAD `http://localhost:5000/professional/E501` Send

Params Authorization Headers (7) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

Body Cookies Headers (7) Test Results (1/1)

{} **JSON** Preview Visualize

200 OK • 23 ms • 237 B • Save Response

1

OPTIONS Request:

REST API basics: CRUD, test & variable / **Post data** Save Share

OPTIONS `http://localhost:5000/professional/E501` Send

Params Authorization Headers (7) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

Body Cookies Headers (8) Test Results (1/1)

Raw Preview Visualize

1 DELETE, GET, HEAD, PATCH, PUT



DELETE:

REST API basics: CRUD, test & variable / **Post data** Save Share

DELETE ▼ Send ▼

Params Authorization Headers (7) **Body** Scripts ▼ Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

Body Cookies Headers (7) Test Results (1/1) 200 OK 23 ms 297 B Save Response

JSON ▼ Preview Visualize ▼

```
1 {
2   "message": "Personal and linked professional records deleted"
3 }
```

MongoDB data:

exp 3 > employee_linked_db > personaldetails Open MongoDB shell

Documents (3) Aggregations Schema Indexes (3) Validation

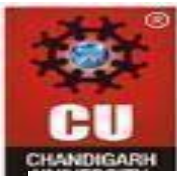
Type a query: { field: 'value' } or [Generate query](#) Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE 25 1 - 3 of 3 ⌂ ⌕ ⌕ ⌕

```
_id: ObjectId('68ca1dcf1fbb98985d76c954')
employeeId: "EMP2025"
firstName: "Sonia"
lastName: "Rao"
email: "sonia.rao@example.com"
phone: "9050843054"
__v: 0
```

```
_id: ObjectId('68ca7ad05a3e64842d719043')
employeeId: "101"
firstName: "Harsh"
lastName: "Jangid"
email: "harshjangid33815@gmail.com"
phone: "9050843054"
__v: 0
```

```
_id: ObjectId('68ca829a092aa2de9e603382')
employeeId: "E101"
firstName: "Harsh"
lastName: "Jangid"
email: "harsh@example.com"
phone: "7776665555"
__v: 0
```



professionaldetails



exp 3 > employee_linked_db > professionaldetails

> Open MongoDB shell

Documents (3) Aggregations Schema Indexes (2) Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find



Options

ADD DATA

EXPORT DATA

UPDATE

DELETE

25

1 - 3 of 3



```
{
  "_id": ObjectId("68ca1e171fbb98985d76c957"),
  "personalId": ObjectId("68ca1dcf1fbb98985d76c954"),
  "employeeId": "EMP2025",
  "department": "Technology",
  "jobTitle": "Software Engineer",
  "dateOfJoining": "2025-09-17T02:33:59.638+00:00",
  "__v": 0
}
```

```
{
  "_id": ObjectId("68ca7dc75a3e64842d719046"),
  "personalId": ObjectId("68ca7ad05a3e64842d719043"),
  "employeeId": "E101",
  "department": "frontend",
  "jobTitle": "Software Engineer",
  "dateOfJoining": "2025-09-17T09:22:15.898+00:00",
  "__v": 0
}
```

```
{
  "_id": ObjectId("68ca82b7092aa2de9e603385"),
  "personalId": ObjectId("68ca829a092aa2de9e603382"),
  "employeeId": "E101",
  "department": "Software Development",
  "jobTitle": "Full Stack Developer",
  "dateOfJoining": "2025-09-17T09:43:19.471+00:00",
  "__v": 0
}
```

Learning Outcomes:

1. Learned to establish a database connection using Mongoose.
2. Designed schemas and models for MongoDB collections.
3. Learned to implement CRUD operations in a Node.js application.
4. Handled proper error handling and response management.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Via		10